A Practical Guide to Knowledge distillation

TOPIC -- KNOWLEDGE DISTILLATION

$$\mathcal{L} = \alpha L_{CE}(y, \sigma(z_s)) + (1 - \alpha) T^2 L_{KD}(\sigma(z_t/T), \sigma(z_s/T))$$

-- 01 --

In this context, a high temperature increases the entropy of the output, therefore providing more information to learn for the distilled model compared to hard targets, and at the same time reducing the variance of the gradient between different records, thus allowing a higher learning rate. If ground truth is available for the transfer set, the process can be strengthened by adding to the loss the cross-entropy between the output y i (x | 1) {\displaystyle y_{i}(\mathbf {x} |1)} of the distilled model computed with t = 1 {\displaystyle t=1} , and the known label y ¯ i {\displaystyle {\bar {y}}_{i}}

-- 02 --

where t {\displaystyle t} is the temperature, a parameter which is set to 1 for a standard softmax. The softmax operator converts the logit values z i (x) {\displaystyle z_{i}(\mathbf {x})} to pseudo-probabilities: higher temperature values generate softer distributions of pseudo-probabilities among the output classes. Knowledge distillation consists of training a smaller network, called the distilled model, on a data set called the transfer set which could correspond to the original training set or consist of new, possibly unlabeled data. A cross-entropy loss function is typically used, computed between the output of the distilled model y (x | t) {\displaystyle \mathbf {y} (\mathbf {x} |t)} and the output of the large model y ^ (x | t) {\displaystyle {\hat {\mathbf {y} }}(\mathbf {x} |t)} on the same record (or the average of the individual outputs, if the large model is an ensemble), using a high value of softmax temperature t {\displaystyle t} for both models:

-- 03 --

y i (x | t) = e z i (x) t ∑ j e z j (x) t {\displaystyle y_{i}(\mathbf {x} |t)={\frac {e^{\frac {z_{i}(\mathbf {x})}{t}}}{\sum _{j}e^{\frac {z_{j}(\mathbf {x})}{t}}}}}

-- 04 --

Do until a desired level of sparsity or performance is reached: Train the network (by methods such as backpropagation) until a reasonable solution is obtained Compute the saliencies for each parameter Delete some lowest-saliency parameters Deleting a parameter means fixing the parameter to zero. The "saliency" of a parameter θ {\displaystyle \theta } is defined as 1 2 (∂ θ 2 L) θ 2 {\displaystyle {\frac {1}{2}}(\partial _{\theta }^{2}L)\theta ^{2}} , where L {\displaystyle L} is the loss function. The second-derivative ∂ θ 2 L {\displaystyle \partial _{\theta }^{2}L} can be computed by second-order backpropagation. The idea for optimal brain damage is to approximate the loss function in a neighborhood of optimal parameter θ ∗ {\displaystyle \theta ^{*}} by Taylor expansion: L (θ) ≈ L (θ ∗) + 1 2 ∑ i (∂ θ i 2 L (θ ∗)) (θ i − θ i ∗) 2 {\displaystyle L(\theta)\approx L(\theta ^{*})+{\frac {1}{2}}\sum _{i}(\partial _{\theta _{i}}^{2}L(\theta ^{*}))(\theta _{i}-\theta _{i}^{*})^{2}} where ∇ L (θ ∗) ≈ 0 {\displaystyle \nabla L(\theta ^{*})\approx 0} , since θ ∗ {\displaystyle \theta ^{*}} is optimal, and the cross-derivatives ∂ θ i ∂ θ j L {\displaystyle \partial _{\theta _{i}}\partial _{\theta _{j}}L} are neglected to save compute. Thus, the saliency of a parameter approximates the increase in loss if that parameter is deleted.

-- 05 --

Mathematical formulation Given a large model as a function of the vector variable x {\displaystyle \mathbf {x} } , trained for a specific classification task, typically the final layer of classification networks is a softmax in the form

-- 06 --

Methods Knowledge transfer from a large model to a small one somehow needs to teach the latter without loss of validity. If both models are trained on the same data, the smaller model may have insufficient capacity to learn a concise knowledge representation compared to the large model. However, some information about a concise knowledge representation is encoded in the pseudolikelihoods assigned to its output: when a model correctly predicts a class, it assigns a large value to the output variable corresponding to such class, and smaller values to the other output variables. The distribution of values among the outputs for a record provides information on how the large model represents knowledge. Therefore, the goal of economical deployment of a valid model can be achieved by training only the large model on the data, exploiting its better ability to learn concise knowledge representations, and then distilling such knowledge into the smaller model, by training it to learn the soft output of the large model.

-- 07 --

In machine learning, knowledge distillation or model distillation is the process of transferring knowledge from a large model to a smaller one. While large models (such as very deep neural networks or ensembles of many models) have more knowledge capacity than small models, this capacity might not be fully utilized. It can be just as computationally expensive to evaluate a model even if it utilizes little of its knowledge capacity. Knowledge distillation transfers knowledge from a large model to a smaller one without loss of validity. As smaller models are less expensive to evaluate, they can be deployed on less powerful hardware (such as a mobile device). There is also a less common technique called Reverse Knowledge Distillation, where knowledge is transferred from a smaller model to a larger one. Model distillation is not to be confused with model compression, which describes methods to decrease the size of a large model itself, without training a new model. Model compression generally preserves the architecture and the nominal parameter count of the model, while decreasing the bits-per-parameter. Knowledge distillation has been successfully used in several applications of machine learning such as object detection, acoustic models, and natural language processing. Recently, it has also been introduced to graph neural networks applicable to non-grid data.

-- 08 --

and under the zero-mean hypothesis ∑ j z j = ∑ j z ^ j = 0 {\displaystyle \sum _{j}z_{j}=\sum _{j}{\hat {z}}_{j}=0} it becomes z i – z ^ i N T 2 {\displaystyle {\frac {z_{i}-{\hat {z}}_{i}}{NT^{2}}}} , which is the derivative of 1 2 (z i – z ^ i) 2 {\displaystyle {\frac {1}{2}}\left(z_{i}-{\hat {z}}_{i}\right)^{2}} , i.e. the loss is equivalent to matching the logits of the two models, as done in model compression.